Python Engineering

WEEK 02 - Data Structures | CLASS 07 - NumPy Arrays | EX15

Cambio oficial de clase: class_06_dictionaries_sets queda completada. Entramos en **class_07_numpy_arrays**, todavía dentro de week_02_data_structures.

EX15 - Analisis de trayectoria con NumPy

Hasta ahora trajectory es una lista de tuplas. NumPy nos permite convertir esa estructura en una **matriz numerica** y operar con columnas completas sin escribir un loop explicito para cada calculo.

```
import numpy as np

trajectory_array = np.array(trajectory)

print(trajectory_array.shape)
# (N, 3)
```

1. Shape: filas y columnas

Mantendremos tu convencion `(h, v, t)`. Cada fila es un estado y cada columna es una variable.

```
# columns
heights    = trajectory_array[:, 0]
velocities = trajectory_array[:, 1]
times      = trajectory_array[:, 2]
```

`:` significa 'todas las filas'. Por eso `trajectory_array[:, 0]` significa: todas las filas, columna 0.

2. Indexacion NumPy

Expresion	Significado
A[0, 0]	fila 0, columna 0
A[0, :]	toda la primera fila
A[:, 0]	toda la primera columna
A[-1, :]	ultima fila completa
A[2:5, 1]	filas 2..4 de la columna 1

3. Diferencia clave: list vs ndarray

```
python_list = [1, 2, 3]
python_list * 2
# [1, 2, 3, 1, 2, 3]

array = np.array([1, 2, 3])
array * 2
# [2, 4, 6]
```

En NumPy las operaciones aritmeticas se aplican elemento a elemento. Esta es una de las razones principales por las que NumPy es la base del calculo cientifico en Python.

4. Vectorizacion

```
specific_energy = config['g'] * heights + 0.5 * velocities**2
```

La ecuacion se aplica de una vez a todos los puntos de la trayectoria. No escribimos un for para cada elemento: NumPy realiza la operacion sobre el array completo.

5. Maximo y posicion del maximo

```
max_h = np.max(heights)
```

```
max_index = np.argmax(heights)
max_state = trajectory_array[max_index, :]

h_max, v_at_max, t_at_max = max_state
```

`np.max()` da el valor maximo. `np.argmax()` da el indice donde se encuentra.

6. Tu mision

```
Create: ex15_numpy_trajectory_analysis.py

1. Reuse your current simulator and obtain trajectory.
2. Convert trajectory to np.array.
3. Print shape and ndim.
4. Extract h, v, t as columns.
5. Print first and last state using NumPy indexing.
6. Compute max altitude using np.max().
7. Find its index with np.argmax().
8. Recover h, v, t at the maximum state.
9. Compute specific mechanical energy for every state.
10. Compare list * 2 with np.array * 2 using a tiny example.
```

7. Validaciones que quiero ver

```
trajectory_array.shape == (len(trajectory), 3)
trajectory_array[0, :] matches initial state
trajectory_array[-1, :] matches final state
np.max(heights) matches previous max_h
v_at_max should be close to 0 m/s
```

8. No queremos todavia

No uses pandas, no hagas graficas y no reescribas Euler con NumPy todavia. Primero quiero que controles arrays, shape, indexacion, slicing y operaciones vectorizadas.

Learning Tracker

Concepto	EX15
np.array()	Nuevo
shape / ndim	Nuevo
indexacion 2D	Nuevo
slicing por filas/columnas	Nuevo
operaciones elemento a elemento	Nuevo
vectorizacion	Nuevo
np.max() / np.argmax()	Nuevo